Linux File System Explorer Using System Calls

KLH University — Empowering Minds

KLH Bachupally Campus | PBL 2026 – 27

Team ID: 08 | Section: S12 | Members: Sindhuja – 2520080045, Jhanasri – 2520090121, Alekhya – 2520090126 | Guide: Raghupathi M

1. Abstract

The Linux File System Explorer is a menu-driven command-line application developed in C to demonstrate Linux system calls for file and directory operations. It helps users manage files and directories, read/write data, view file information and delete files. The project also covers process management, inter-process communication (IPC), memory management, and thread synchronization. It provides practical exposure to core operating system concepts and helps in understanding how user-space applications interact with the Linux kernel.

2. Problem Statement & Objectives

Problem:

Managing files, processes and system resources in a Linux environment requires knowledge of system calls and OS concepts. Users need a practical tool to understand and interact with these features.

Objectives:

1. Implement file and directory operations using system calls.
2. Demonstrate process management, IPC, memory and synchronization.
3. Provide a simple and interactive command-line interface.

Scope:

Users: Students and learners | **Functions:** File ops, process, IPC, memory, threads | **OS Concepts:** System calls, processes, IPC, memory, synchronization | **Project Boundary:** Linux environment, C programming.

3. Existing vs Proposed System

Aspect	Basic / Existing Approach	Proposed Project
Process handling	Manual or limited process handling	Create/manage processes using fork(), exec(), wait()
Communication	No / limited communication	Pipe, FIFO, signal based IPC
Resource handling	Limited file/resource management	Efficient file & directory operations using system calls
Error handling	Common failure without proper message	Validation and error-handling mechanism
Key gap	Lacks integration of multiple OS concepts	Unified system with files, processes, IPC, memory, threads

4. Hardware & Software Requirements

Hardware

Processor	Intel Core i3 or above
RAM	4 GB or above
Storage	10 GB free space

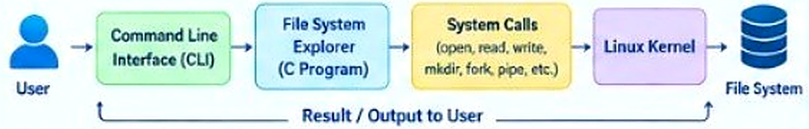
Software

Operating System	Linux / Ubuntu
Programming Language	C
Compiler	GCC
Interface	CLI
APIs	Linux/POSIX System Calls
Development Tool	VS Code / Linux Terminal

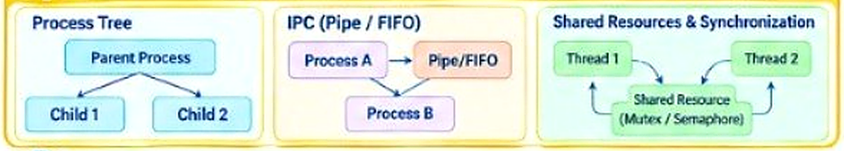
5. Tools & Technologies

C Programming | Linux/Ubuntu Platform | System Calls (Processes, IPC, Files, Memory, Sync) | GCC Compiler | Make Tool | Git & GitHub Version Control | VS Code IDE

6. System Workflow / Architecture



7. Process, IPC & Resource Design



11. Conclusion

The Linux File System Explorer demonstrates basic file and directory operations using system calls. It helps us understand how user applications interact with the Linux kernel. Through this project, we gained practical knowledge of file handling, input/output, directories, file descriptors, process management, IPC, memory management and thread synchronization.

8. Individual Contributions

Roll Number	Student Name	Contribution
2520080045	Sindhuja	Project design, menu-driven interface, file creation, reading and writing.
2520090121	Jhanasri	Directory operations using mkdir(), opendir(), readdir(), closedir().
2520090126	Alekhya	File information and deletion using stat(), and unlink(), error handling, testing, documentation, GitHub management.

9. Implementation Screenshots

```
Linux File System Explorer
  1. List files
  2. Create file
  3. Read file
  4. Write file
  5. Delete file
  6. Rename file
  7. Create directory
  8. Remove directory
  9. File information
 10. Exit

 Enter your choice: 1
```

Figure 1: Main Menu Interface

```
Enter file name: sample.txt

File Information:
Size         : 1024 bytes
Type         : Regular File
Permissions  : -rw-r--r--
Accessed     : 2026-09-02 10:24:33
Modified     : 2026-09-02 10:20:17
Created      : 2026-09-02 10:20:17

Enter your choice: 9
```

Figure 2: File Information (stat())

10. Results & Testing

```
$ ./file_explorer
$ Created file: test.txt
$ File written successfully.
$ File contents:
$ Hello Linux!
$ File deleted successfully.
```

Testing

- ✓ Normal cases (create, read, write, delete)
- ✓ Boundary cases (empty file, large file)
- ✓ Invalid input (wrong file name)
- ✓ Process/IPC behavior (pipe, FIFO, signal)
- ✓ Error handling (permission denied, not found)

12. Future Scope

- Add GUI for better user experience.
- Implement advanced file search and filtering.
- Extend IPC with shared memory and message queues.
- Add file permission modification and ownership features.
- Improve performance and testing automation.